

FUP - Oral Exam

neslusam and question providers

Contents

Introduction	1
Functional Programming	1
Racket	2
λ -Calculus	3
Haskell	8

⚠ **TRIGGER WARNING - M*NAD** ⚠ - The text below contains repeated, explicit occurrences of the “M-Word”. If you are currently recovering from trying to understand what a m*nad actually is, further review of this material is not recommended..

Introduction

[V] - Tomáš Votroubek , [H] - Rostislav Horčík , [Z] - Matěj Zorek **Your examiner is random, your previous lecturer doesn’t make a difference.**

There might be a newer version of this document, check out this [github repository](#)

Functional Programming

[Z] *What is pure/Higher-order function + examples of each possible combination?*

Solution:

Pure function is like a mathematical function. Pure function is a **deterministic function without any side effects** (e.g., no modifying global variables, no writing to files)

Higher-order function: function taking **other functions as arguments** or **returning a function** or both is called a higher-order function.

order \ purity	Pure	Impure
First-Order	circle_area $r = \pi * r * r$	print x / getLine
Higher-order	map / filter / foldl / apply	mapM / modify (State monad)

[Z] *List types of recursions based on branching factor and provide examples. Classify head/ tail recursion and discuss theirs branching factor.*

Solution:

Recursion is split into **linear/tree recursion**.

Linear recursion makes **one** recursive call per execution step. Example: **Factorial**

Tree (Non-linear) recursion makes **two or more** recursive calls per execution step. Example: **Fibonacci**

In **head recursion**, the recursive call is made **before** the rest of the function's operations are completed. Head recursion is **usually non-linear**, because linear recursion is faster with tail recursion.

Example:

```
factorialHead 0 = 1
factorialHead n = n * factorialHead (n - 1)
```

In **tail recursion** the recursive call is the **very last** operation executed by the function. It often uses an **accumulator**. Tail recursion is always **linear**.

Example:

```
factorialTail 0 acc = acc
factorialTail n acc = factorialTail (n - 1) (n * acc)
```

Racket

[V] *How to implement infinite stream of nats/ones*

1. *using stream functions?*
2. *without using stream functions?*

Solution:

1. Example - stream of natural numbers using stream-cons

```
(define (stream_nats n)
  (stream-cons n (stream_nats (+ n 1))))
```

- 2.

```
(define (nats_lambda n)
  (cons n (lambda () (nats_lambda (+ n 1)))))

;;for extra points use syntactic sugar thunk ;)

(define (nats_thunk n)
  (cons n (thunk (nats_thunk (+ n 1)))))
```

[V] *Describe how does foldl/foldr work, which parametres are used and what is the difference between foldl and foldr.*

Solution:

Foldl/r collapses a list into a single value, combining its elements with an accumulator from left to right (foldl) / from right to left (foldr).

Example:

```
(foldl append '() '((1 2) (3 4) (5 6)))  
;; Output: '(5 6 3 4 1 2)
```

```
(foldr append '() '((1 2) (3 4) (5 6)))  
;; Output: '(1 2 3 4 5 6)
```

λ -Calculus

[Z] List and describe terms in syntax of λ -calculus.

Solution:

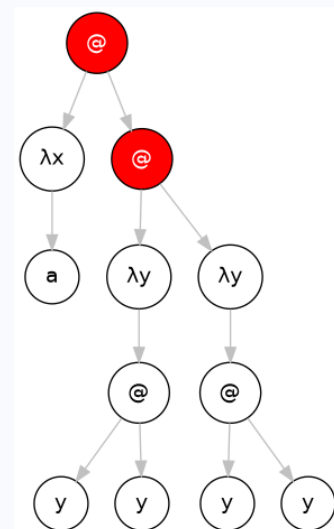
The syntax of lambda calculus has only three types of terms: a **variable** (denoted by lowercase letters $x, y, z, \dots$), the **abstraction** of a variable from a term defining a function (denoted as $\lambda x.t$), and the **application** of a term to a term (e.g. xy , $(\lambda x.y)y$, $\dots$).

[H/Z] What is a redex?

Solution:

A **redex** (reducible expression) is any term that **matches abstraction to an argument**. It is similar to having a function (abstraction) that already got the argument and can be evaluated. It is really easy to find in an abstract syntax tree. Redex is **every application**, where the left argument is an abstraction (lambda symbol).

Example: This AST has all the redexes marked red.



[H] What is a reduction? List and describe different types of reductions and their usage.

Solution:

Reduction is the step-by-step process of **evaluating** or simplifying an expression **until it cannot be simplified any further**.

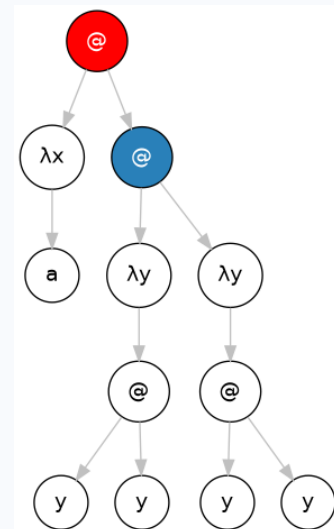
The two essential transformation rules in lambda calculus are:

1. **α -conversion** - The process of **renaming a bound variable** (and all its occurrences inside the function body). Used to **prevent naming conflicts**
2. **β -reduction** - The actual computation step. It applies a function to an argument by **substituting the argument for all free occurrences of the bound variable inside the function body**.

There are **two** different strategies to evaluation:

1. **Left outermost** (Lazy evaluation/normal order) - furthest to the left and closest to the outside of the expression (**not contained inside any other redex**).
2. **Left innermost** (Eager evaluation/applicative order) - furthest to the left but nested the deepest inside the expression (**it contains no other redexes inside itself**).

Example: **Red** - left outermost , **blue** - left innermost

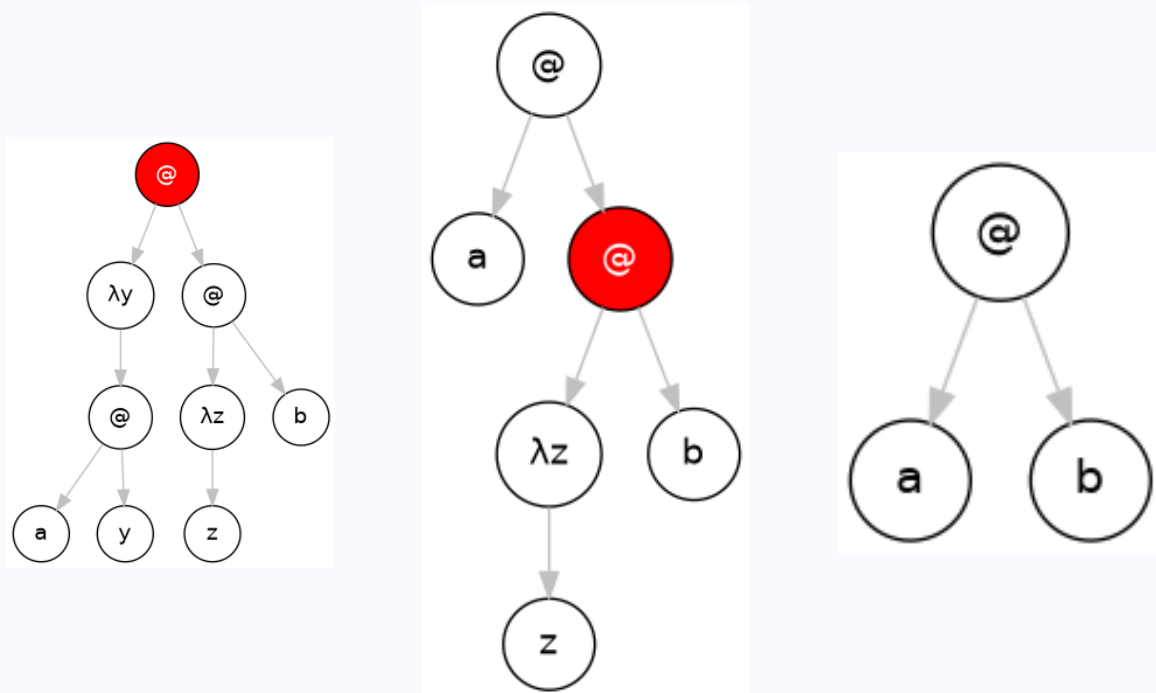


[V] What is a normal form and how to transform a λ -expression to its normal form?

Solution:

Expression is in a **normal form** if it cannot be simplified any further – it contains no redexes. You can transform an expression into its normal (if it exists) form using **repeated β -reductions** until there are no redexes. The preferred evaluation order is **left outermost** (see Church-Rosser theorems to learn why).

Example: transformation of $(\lambda y. ay)((\lambda z. z)b)$



Useful tips:

Be aware of name collision! Perform α -conversion before reducing to keep variable names unique.

Some expression do not converge and do not have a normal form! Example: $(\lambda x.xx)(\lambda x.xx)$

[Z] What is the difference between bound and free variable? Which operation is done on them?

Solution:

Bound variable is inside of an abstraction, free is not.

Possible operations(probably what the teacher meant):

Bound variable - α -conversion - you can rename lambda and each instance in the body

Free variable - β -reduction - argument of a redex destroys the bound variables, only keeping the free ones.

[H] What are the Church-Rosser theorems?

Solution:

In the answers above, we have talked about different evaluation orders/strategies. That

gives rise to two natural questions. For a given term, do all evaluation orders produce the same normal form? Is the process of repeated β -reduction guaranteed to terminate for all evaluation orders if the normal form exists? The Church-Rosser theorems give us the answer.

Explicitly, they state that

1. Normal forms are independent of the evaluation strategy – a lambda term cannot be reduced to two different normal forms.
2. Normal order (lazy) evaluation always finds a normal form if it exists.

Informally, if we are to reduce a lambda term, normal order (lazy) evaluation is the way to go. It is guaranteed to produce a result (if the normal form exists) and the result is the same no matter the evaluation order.

[H] *how to represent True/False in λ -Calculus?*

Solution:

T = $\lambda x \lambda y. x$

F = $\lambda x \lambda y. y$

Boolean is interpreted as a **decision between 2 options** and you either pick the first one (T) or the second one (F)

[H] *How to represent number in λ -Calculus? How to represent addition of 2 numbers?*

Solution:

Number n is represented as applying function f to starting x n-times.

$0 = \lambda f. \lambda x. x$ $1 = \lambda f. \lambda x. f(x)$ $2 = \lambda f. \lambda x. f(f(x))$

addition is defined as:

ADD = $\lambda m. \lambda n. \lambda f. \lambda x. m f (n f (x))$

Example: $\text{ADD } 1 \ 2 = \lambda f. \lambda x. 1 f (2 f (x))$ - **Note! We already defined what 1 and 2 means.**

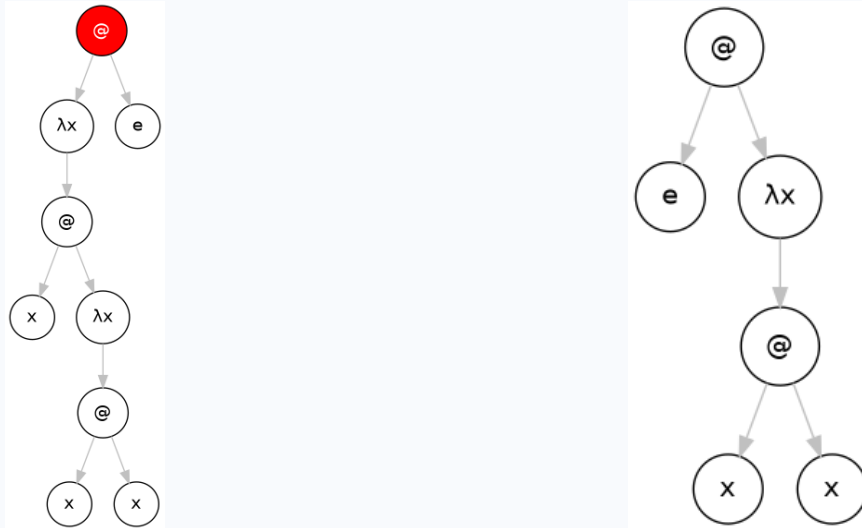
Argument of 1 f is $(2 f (x)) = f (f (x))$

We are left with $\lambda f. \lambda x. 1 f (f (f (x)))$, which is transformed back to $\lambda f. \lambda x. f (f (f (x))) = \mathbf{3}$

[H] *reduce this expression $(\lambda x. x(\lambda x. xx))e$ to its normal form.*

Solution:

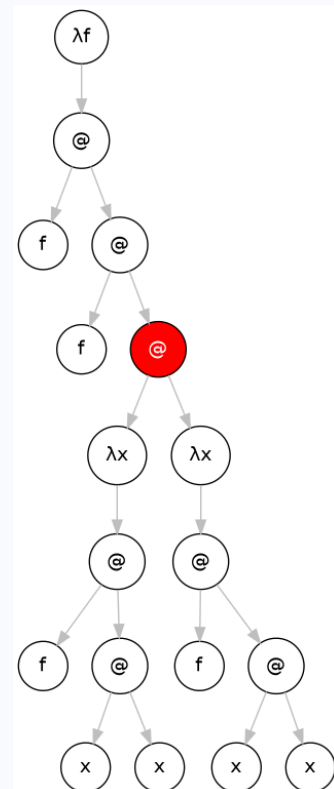
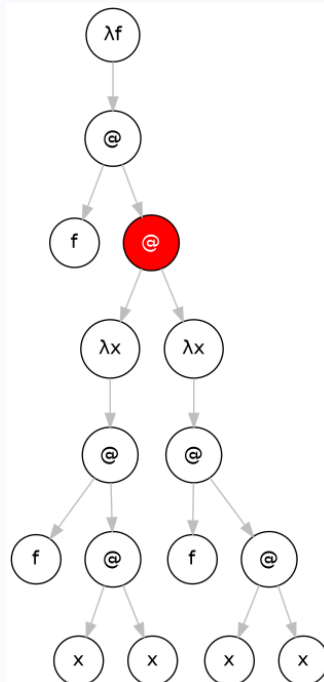
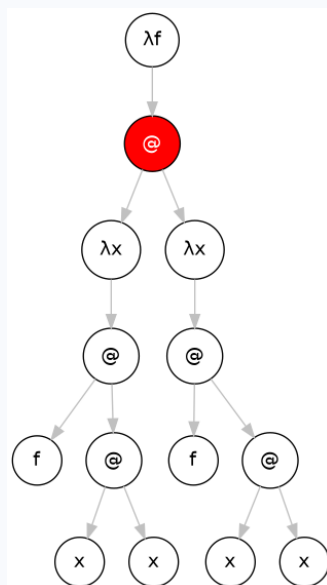
This reduction is tricky, because there are **two abstractions with the same name**, I would generally recommend to rename one to prevent mistakes from happening.



[V] Find all free/bound vars, redexes in $(\lambda f. (\lambda x. f (x x)) (\lambda x. f (x x)))$. Find normal form.

Solution:

All variables are bound and there is only one redex (marked red). After a few steps, it is obvious that the term **grows infinitely and thus doesn't have a normal form**.



Haskell

[V] Describe signature of function map.

Solution:

```
map :: (a -> b) -> [a] -> [b]
```

[V] Describe signature of function foldl/r.

Solution:

```
foldr :: Foldable t => (a -> b -> b) -> b -> t a -> b
foldl :: Foldable t => (b -> a -> b) -> b -> t a -> b
```

[H] What is a typeclass?

Solution:

Typeclass is a sort of interface that defines some behavior. Typeclasses enable functions to work on different types while supporting certain operations.

For a custom data type, you can usually derive some typeclasses: `Eq`, `Ord`, `Enum`, `Bounded`, `Show`, `Read`. If this is not the case, you need to implement it yourself ;).

[H] How to implement polymorphic typeclass, which adds 1 to an `Int` and prints out length for `String`?

Solution:

You need to create a polymorphic typeclass with a single method that **returns IO monad**.

```
class Magnesium a where
  wheatBran :: a -> IO ()

instance Magnesium Int where
  wheatBran n = do
    let next = n + 1
    print next

instance Magnesium String where
  wheatBran str = do
    let len = length str
    print len
```

[V] What is `Functor` typeclass? Which functions define it?

Solution:

Functor is any data type which can be mapped over. It enables applying pure functions without changing the structure.

Core function: `fmap :: (a -> b) -> f a -> f b` (or the `<$>` operator)

Example:

```
addOne = fmap (+1)

addOne (Map.fromList [('a',1), ('b',2)])
--Output: Map.fromList [('a',2), ('b',3)]
```

[V] What is a `Applicative` typeclass? Which functions define it?

Solution:

`Applicative` typeclass **extends Functor** typeclass. With a `Functor`, you use `<$>` to map a normal function over a wrapped value. With a **`Applicative`** typeclass, you can **apply a function that is already wrapped to a value that is also wrapped** using `<*>`, called `apply` or `ap`.

Another function of applicative typeclass is to **wrap in the simplest way possible** into a certain data type, using the function **pure**.

Core functions:

```
pure :: a -> f a
```

```
(<*>) :: f (a -> b) -> f a -> f b
```

Examples:

```
-- fmap fails here, because the function is wrapped
```

```
Just (+10) <$> Just 5 -- Result: Error
```

```
Just (+10) <*> Just 5 -- Result: Just 15
```

```
Nothing <*> Just 5 -- Result: Nothing
```

```
pure 5 :: Maybe Int -- Output: Just 5
```

```
pure 5 :: [Int] -- Output: [5]
```

[V] *What is a Monad? Which functions define it?*

Solution:

A Monad **extends Applicative** typeclass. Monads allow to **chain dependent operations**, meaning the output of **step A directly determines what step B will look like**. In contrast, Applicative executes a series of independent operations that are ignorant to each other until they are combined at the very end.

Core Functions:

1. **bind** (written as `>=>`): It feeds a boxed value to a function that returns a new box, without double-boxing. In the `do` notation, it is written with an arrow `<-`

```
(>=>) :: m a -> (a -> m b) -> m b
```

Example:

```
Just 10 >=> (\x -> Just (x + 1)) --Output: Just 11 (Unpacks 10, adds 1, repacks)
```

```
Nothing >=> (\x -> Just (x + 1)) --Output: Nothing
```

```
Just 10 <*> (\x -> Just (x + 1)) --Output: Error (Applicative is not enough)
```

2. **>> operator**: `>>` chains two monadic actions together by running the first, discarding its value, and then running the second. This seems useless at first, but under the hood, every time you write a line in a `do` block without using `<-`, Haskell translates it to `>>` under the hood.

```
(>>) :: m a -> m b -> m b
```

Example:

```
--This is equivalent:
```

```
main = do
```

```

putStrLn "Hello"
putStrLn "World"

--to this:
main = putStrLn "Hello" >> putStrLn "World"

```

3. **return**: the **same function as pure** in Applicative typeclass

```
return :: a -> m a
```

Example from 1. rewritten with do notation.

```

successExample :: Maybe Int
successExample = do
  x <- Just 10
  return (x + 1)

```

[H] *What is a State Monad, what is it good for? How to implement bind for State Monad?*

Solution:

State Monads simulate mutable state in pure functional programming. State is just a function that takes an initial state s , and returns a tuple containing the result a and a new state s .

```
State s a = s -> (a, s)
```

Even though a student has only been asked about **bind**, I highly recommend looking into `State.hs` provided in the official page. In my opinion, the question about **bind** opens possibilities to be asked about different functions.

```

newtype State s a = S { runState :: s -> (a, s) }

instance Monad (State s) where
  -- (>=) :: State s a -> (a -> State s b) -> State s b
  stx >= f = S (\s -> let (x, s') = runState stx s
                      in runState (f x) s')

```

[H] *What are semigroups, monoids¹ and foldables?*

Solution:

Given a list of `Ints`, we may want to fold such a list. One way to do so is to use the `+` operation to combine the individual elements of the list. More abstractly, to perform a fold, we need a set, `Int`, and an operation that set, $+: \text{Int} \times \text{Int} \rightarrow \text{Int}$. Having this operation be associative (which `+` is) makes `foldl` and `foldr` produce the same result. This idea can be further abstracted and formalized as a semigroup.

¹**Monoids** are not to be confused with **monads**. That said, a monad is just a monoid in the category of endofunctors but one need not understand that in order to understand monoids and monads from a programming point of view.

A semigroup $\mathbb{S} = (S, \odot)$ is a set S and an operation $\odot : S \times S \rightarrow S$ satisfying associativity,

$$(x \odot y) \odot z = x \odot (y \odot z)$$

for all $x, y, z \in S$. In Haskell, a semigroup is defined as

```
class Semigroup a where
  (<>) :: a -> a -> a
```

As usual, this definition does not explicitly require associativity as Haskell's type system is not strong enough to express it.

Another thing worth noticing is that we want to be able to fold an empty list which requires a “default” value. We also expect this value to not influence the folding of a nonempty list. For natural numbers we use 0 as such value which is the identity element w.r.t. the operation $+$. This idea may be abstracted as having an identity element u w.r.t. the $\odot$ operation.

A monoid $\mathbb{M} = (M, \odot, u)$ is the semigroup $(M, \odot)$ satisfying

$$u \odot x = x = x \odot u,$$

for all $x \in M$. In Haskell, a monoid is defined in the following way

```
class Semigroup a => Monoid a where
  mempty :: a
```

The `mempty` here represents the unit u from the mathematical definition from above.

So far, we have abstracted the way a fold aggregates but we are yet to abstract the traversal of a list. The idea here is that if we want to fold a data structure whose elements are of type A using the monoid $\mathbb{M} = (M, \odot, u)$, all we need is a function $f : A \rightarrow M$. To give an example, given a list of type `[a]` and a `Monoid m`, to fold such a list, we only need a function `f :: a -> m`. The function `f` allows us to convert all elements of the list to `ms` and then fold those using the operation of the monoid and its unit. This idea is formalized in the definition of a `Foldable`:

```
type Foldable :: (* -> *) -> Constraint
class Foldable t where
  foldMap :: Monoid m => (a -> m) -> t a -> m
  ...
```

The foldable class provides default implementations for many other useful functions besides `foldMap` such as `fold`, `length` etc.

You are all done!

